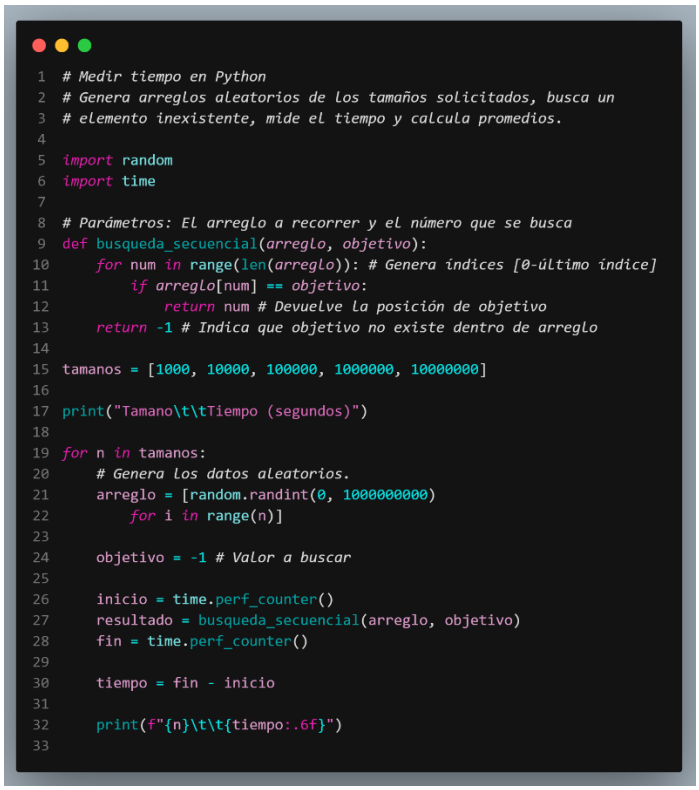
Búsqueda secuencial en Python



```python
# Medir tiempo en Python
# Genera arreglos aleatorios de los tamaños solicitados, busca un
# elemento inexistente, mide el tiempo y calcula promedios.

import random
import time

# Parámetros: El arreglo a recorrer y el número que se busca
def busqueda_secuencial(arreglo, objetivo):
    for num in range(len(arreglo)): # Genera índices [0-último índice]
        if arreglo[num] == objetivo:
            return num # Devuelve la posición de objetivo
    return -1 # Indica que objetivo no existe dentro de arreglo

tamanos = [1000, 10000, 100000, 1000000, 10000000]

print("Tamano\t\tTiempo (segundos)")

for n in tamanos:
    # Genera los datos aleatorios.
    arreglo = [random.randint(0, 1000000000)
        for i in range(n)]

    objetivo = -1 # Valor a buscar

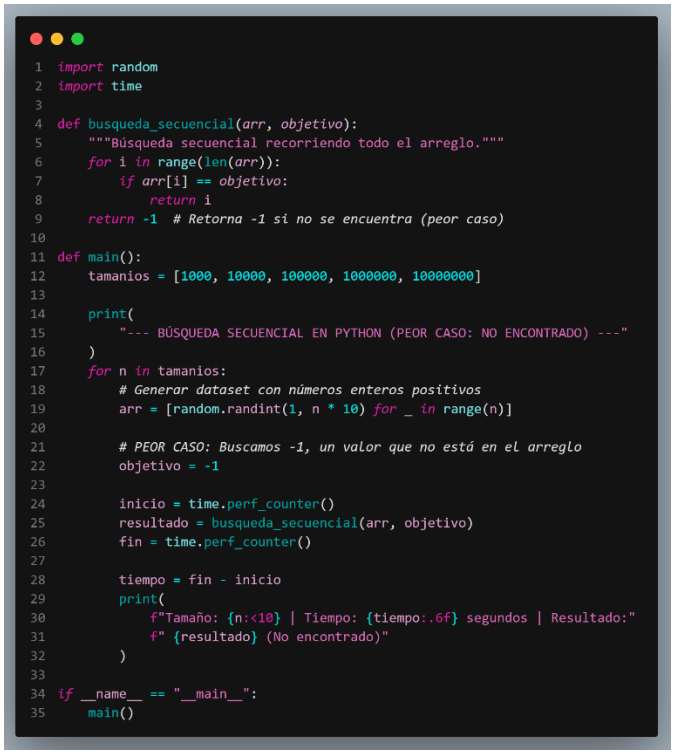
    inicio = time.perf_counter()
    resultado = busqueda_secuencial(arreglo, objetivo)
    fin = time.perf_counter()

    tiempo = fin - inicio

    print(f"{n}\t\t{tiempo:.6f}")

```

Imagen 1. Código búsqueda secuencial en Python creado por nosotros.



```python
import random
import time

def busqueda_secuencial(arr, objetivo):
    """Búsqueda secuencial recorriendo todo el arreglo."""
    for i in range(len(arr)):
        if arr[i] == objetivo:
            return i
    return -1  # Retorna -1 si no se encuentra (peor caso)

def main():
    tamanios = [1000, 10000, 100000, 1000000, 10000000]

    print(
        "--- BÚSQUEDA SECUENCIAL EN PYTHON (PEOR CASO: NO ENCONTRADO) ---"
    )
    for n in tamanios:
        # Generar dataset con números enteros positivos
        arr = [random.randint(1, n * 10) for _ in range(n)]

        # PEOR CASO: Buscamos -1, un valor que no está en el arreglo
        objetivo = -1

        inicio = time.perf_counter()
        resultado = busqueda_secuencial(arr, objetivo)
        fin = time.perf_counter()

        tiempo = fin - inicio
        print(
            f"Tamaño: {n:<10} | Tiempo: {tiempo:.6f} segundos | Resultado:"
            f" {resultado} (No encontrado)"
        )

if __name__ == "__main__":
    main()
```

Imagen 2. Código búsqueda secuencial en Python creado con IA.

	Código propio	Código hecho por IA
Núm. elementos	Tiempo (Segundos)	
1,000	0.000053	0.000044
10,000	0.000439	0.000407
100,000	0.005894	0.009926
1,000,000	0.045692	0.050949
10,000,000	0.599248	0.527818

Tabla 1.Tabla de pruebas Python búsqueda secuencial.

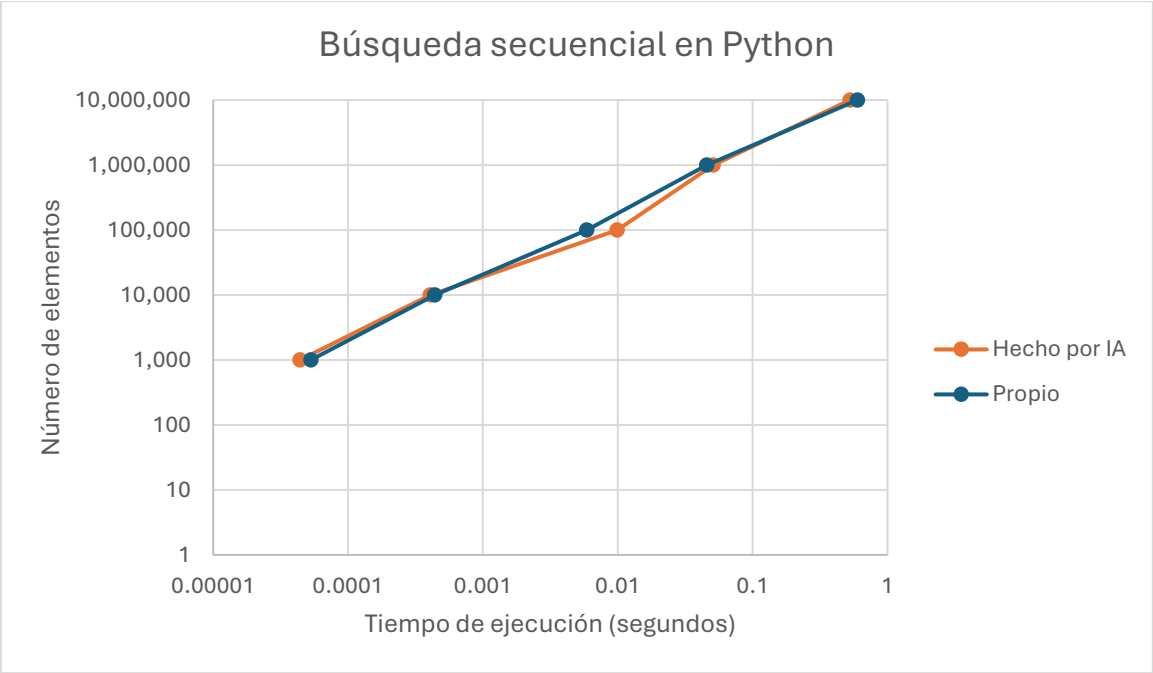
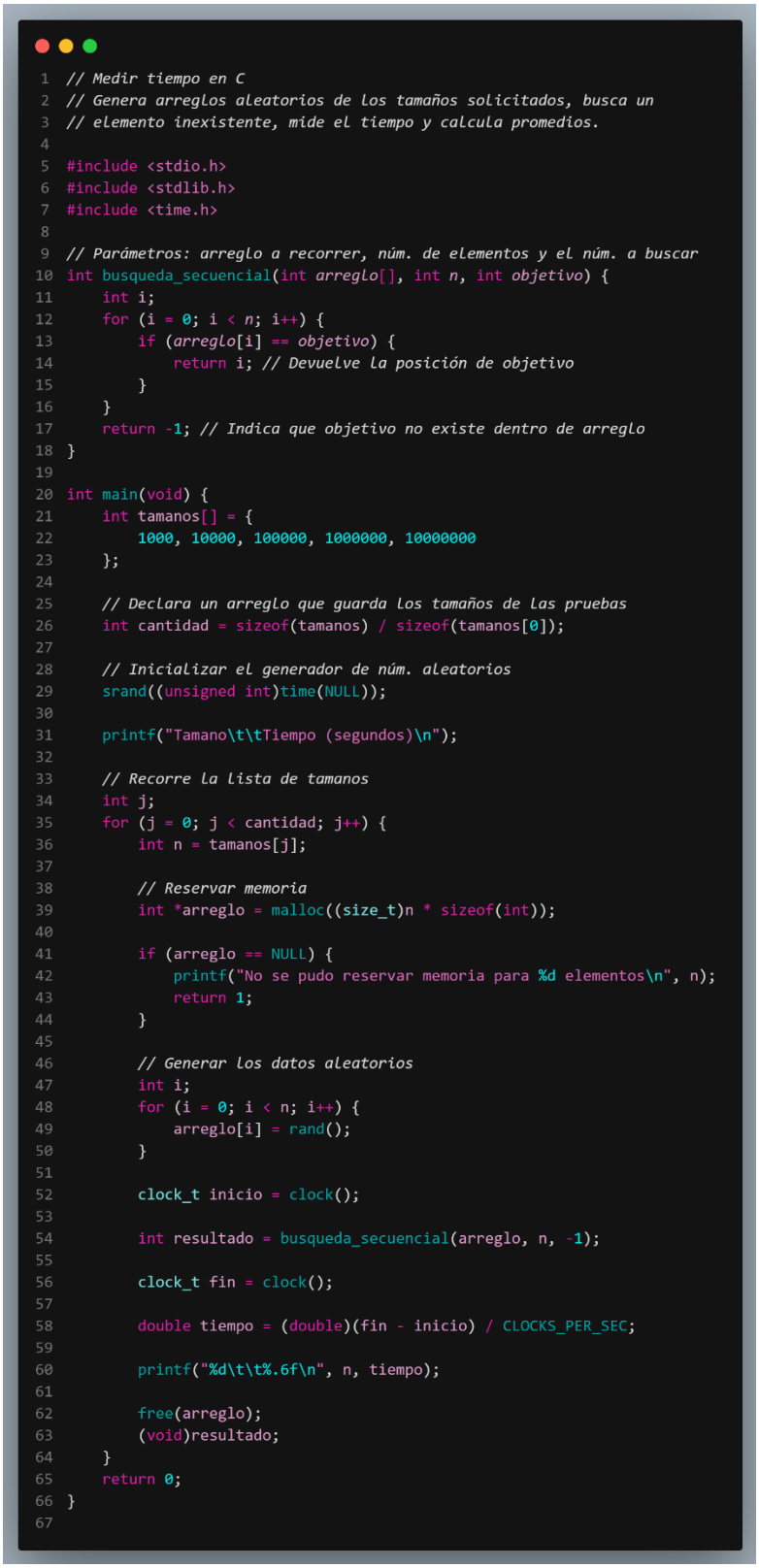


Ilustración 1. Comparación de tiempos en Python.

Búsqueda secuencial en C



```c
// Medir tiempo en C
// Genera arreglos aleatorios de los tamaños solicitados, busca un
// elemento inexistente, mide el tiempo y calcula promedios.

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Parámetros: arreglo a recorrer, núm. de elementos y el núm. a buscar
int busqueda_secuencial(int arreglo[], int n, int objetivo) {
    int i;
    for (i = 0; i < n; i++) {
        if (arreglo[i] == objetivo) {
            return i; // Devuelve la posición de objetivo
        }
    }
    return -1; // Indica que objetivo no existe dentro de arreglo
}

int main(void) {
    int tamanos[] = {
        1000, 10000, 100000, 1000000, 10000000
    };

    // Declara un arreglo que guarda los tamaños de las pruebas
    int cantidad = sizeof(tamanos) / sizeof(tamanos[0]);

    // Inicializar el generador de núm. aleatorios
    srand((unsigned int)time(NULL));

    printf("Tamano\t\tTiempo (segundos)\n");

    // Recorre la lista de tamanos
    int j;
    for (j = 0; j < cantidad; j++) {
        int n = tamanos[j];

        // Reservar memoria
        int *arreglo = malloc((size_t)n * sizeof(int));

        if (arreglo == NULL) {
            printf("No se pudo reservar memoria para %d elementos\n", n);
            return 1;
        }

        // Generar los datos aleatorios
        int i;
        for (i = 0; i < n; i++) {
            arreglo[i] = rand();
        }

        clock_t inicio = clock();

        int resultado = busqueda_secuencial(arreglo, n, -1);

        clock_t fin = clock();

        double tiempo = (double)(fin - inicio) / CLOCKS_PER_SEC;

        printf("%d\t\t%.6f\n", n, tiempo);

        free(arreglo);
        (void)resultado;
    }
    return 0;
}

```

Imagen 3. Código búsqueda secuencial en C creado por nosotros.



```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Búsqueda secuencial optimizada
long long busqueda_secuencial(const int *arr, long long n, int objetivo) {
    long long i;
    for (i = 0; i < n; i++) {
        if (arr[i] == objetivo) {
            return i;
        }
    }
    return -1; // Retorna -1 si no lo encuentra (peor caso)
}

int main() {
    long long tamanios[] = {1000, 10000, 100000, 1000000, 10000000};
    int num_tamanios = sizeof(tamanios) / sizeof(tamanios[0]);

    srand(time(NULL));

    printf("--- BÚSQUEDA SECUENCIAL EN C (PEOR CASO: NO ENCONTRADO) ---\n");
    int t;
    for (t = 0; t < num_tamanios; t++) {
        long long n = tamanios[t];

        int *arr = (int *)malloc(n * sizeof(int));
        if (arr == NULL) {
            printf("Error de asignación de memoria\n");
            return 1;
        }

        // Generar dataset con números positivos aleatorios
        long long i;
        for (i = 0; i < n; i++) {
            arr[i] = rand() % 1000000 + 1;
        }

        // PEOR CASO: Buscamos -1, un valor garantizado de no existir en el arreglo
        int objetivo = -1;

        clock_t inicio = clock();
        long long resultado = busqueda_secuencial(arr, n, objetivo);
        clock_t fin = clock();

        double tiempo = (double)(fin - inicio) / CLOCKS_PER_SEC;

        printf("Tamaño: %-10lld | Tiempo: %.6f segundos | Resultado: %lld (No encontrado)\n", n, tiempo, resultado);

        free(arr);
    }

    return 0;
}
```

Imagen 4. Código búsqueda secuencial en C creado con IA.

	Código propio	Código hecho por IA
Núm. elementos	Tiempo (Segundos)	
1,000	0.000000	0.000000
10,000	0.000000	0.000000
100,000	0.000000	0.000000
1,000,000	0.003000	0.012000
10,000,000	0.035000	0.041000

Tabla 2. Tabla de pruebas C búsqueda secuencial.

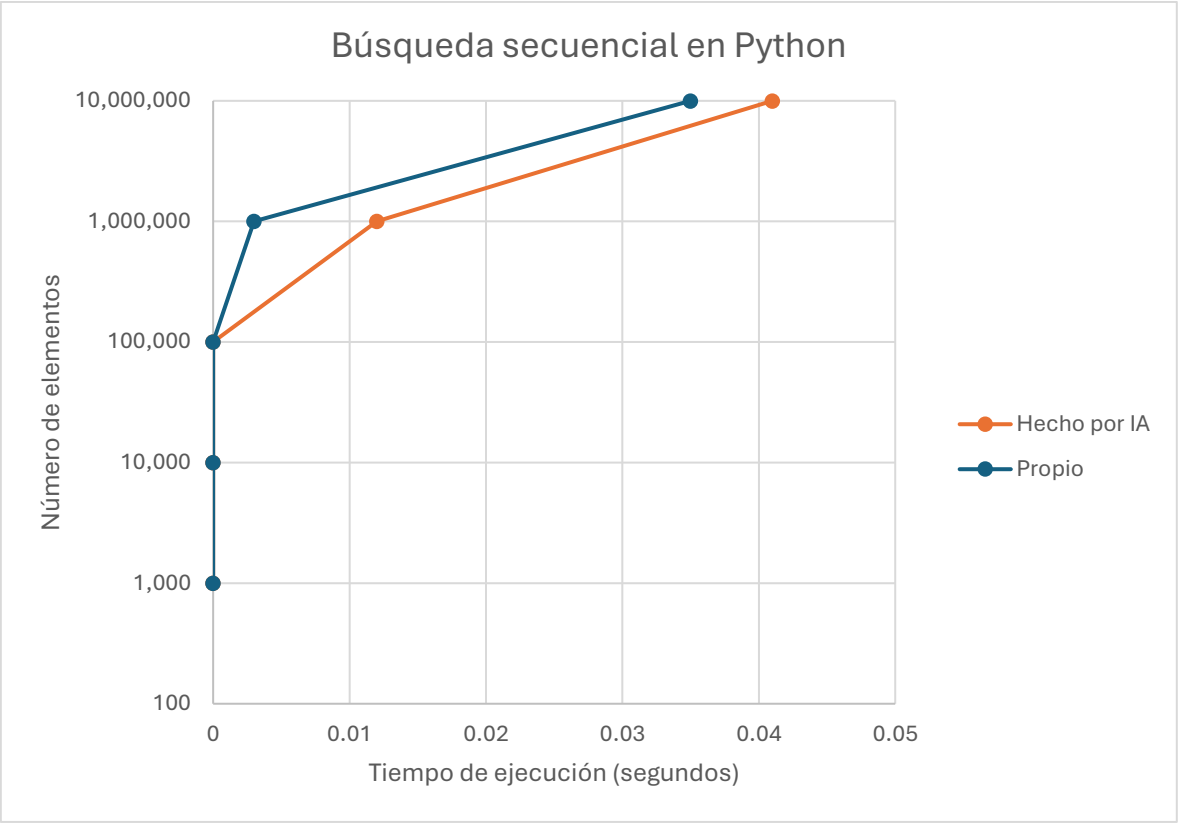


Ilustración 2. Comparación de tiempos en C.